
title: "Technical Documentation & Architecture Specification"
author: "Project Hyperion"
date: "Final Edition"

<div align="center">

Project Hyperion

Technical Documentation & Core Logic Specification

Problem Statement 02 (PS-02) & Problem Statement 03 (PS-03)

FINAL READABLE EDITION

</div>

Executive Summary

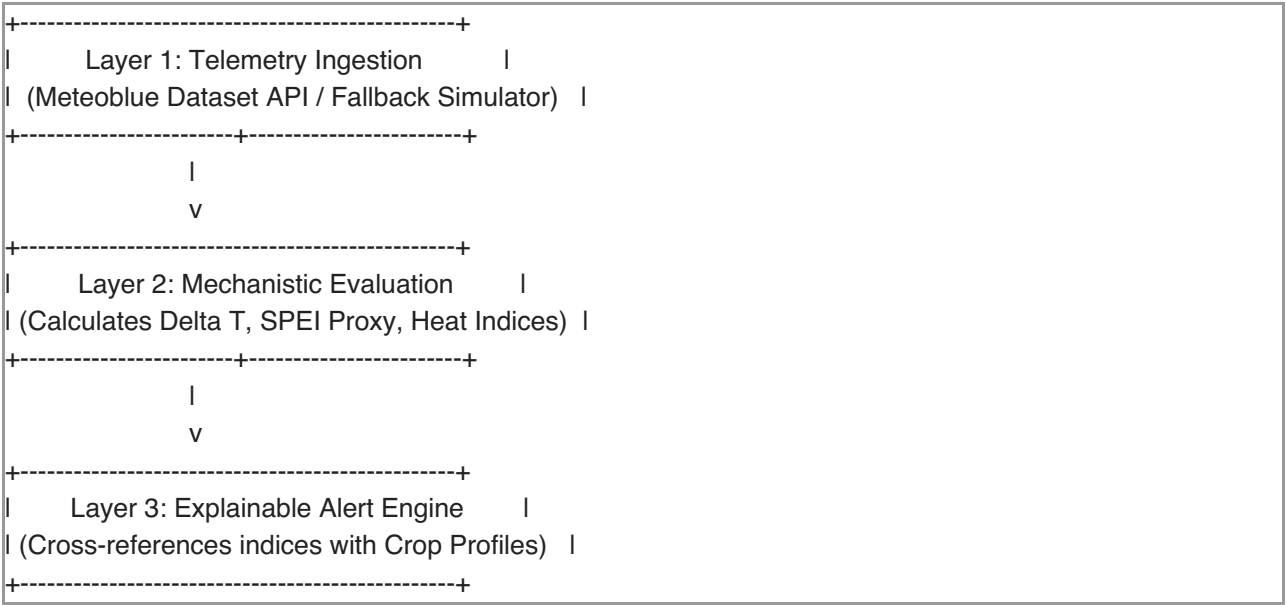
Project Hyperion is a production-ready agricultural intelligence platform engineered for Syngenta Biologicals. By unifying real-time weather telemetry with hybrid mechanistic stress models and generative AI intent extraction, Hyperion delivers predictive and personalized biological intervention strategies.

This document serves as the authoritative technical reference for the data ingestion pipelines, mathematical algorithms, API routing, and deterministic decision matrices that power **PS-02 (Climate Stress Early Warning)** and **PS-03 (CropFit - Personalised Biological Product Advisor)**.

Part 1: Problem Statement 02 (PS-02) — Climate Stress Early Warning

1.1 Architectural Overview & Data Flow

PS-02 functions as a 14-day prognostic engine. The architecture ensures high-fidelity forecasting by cascading through three distinct processing layers.



1.2 Telemetry Ingestion & API Logic (data_ingestion.py)

The platform prioritizes live data from **Meteoblue's Dataset API**. To parse the JSON payload accurately, the engine extracts deeply nested meteorological parameters via their specific Meteoblue API codes:

- **Code 11:** Maximum Temperature ($T_{\max}$)
- **Code 52:** Relative Humidity (RH)
- **Code 61:** Precipitation (mm)
- **Code 32:** Wind Speed (m/s)

Calculated Telemetry:

During ingestion, the backend instantly calculates agronomic derived variables:

- **Delta T (ΔT):** Used to evaluate spraying conditions.
$$\Delta T \approx T_{\max} - \left(T_{\max} \times \frac{RH}{100}\right)$$
- **SPEI Proxy:** A localized water-balance equation predicting drought intensity.
$$\text{SPEI} = \frac{\text{Actual Precip} - \text{Expected Precip}}{\max(\text{Expected Precip}, 1)}$$

Fail-Safe Strategy: If the Meteoblue API rate-limits or fails, the engine seamlessly fails over to a deterministic CE Hub Mock Engine which simulates a standard 14-day agricultural forecast using statistical normal distributions around seasonal baselines.

1.3 Hybrid Mechanistic Ensemble Math (ensemble_model.py)

The core of PS-02 is the HybridEnsembleModel, which scores environmental stress on a continuous [0.0, 1.0] scale.

1. Heat Stress Scoring

The model calculates a base thermal penalty when $T_{\max}$ exceeds the crop's absolute tolerance threshold (e.g., 38°C for Cotton).

$$\text{Base Penalty} = (T_{\max} - T_{\text{critical}}) \times 0.1$$

Growth Stage Multiplier: If the crop is in a vulnerable phenological stage (e.g., Flowering), the base penalty is heavily amplified ($1.5 \times$).

2. Drought Stress Scoring

Utilizing the SPEI proxy calculated during ingestion, the engine maps moisture deficit to a drought severity curve. If $\text{SPEI} < -0.5$ (severe deficit), the Drought_Score scales non-linearly up to 1.0\$.

3. Compound Stress Amplification

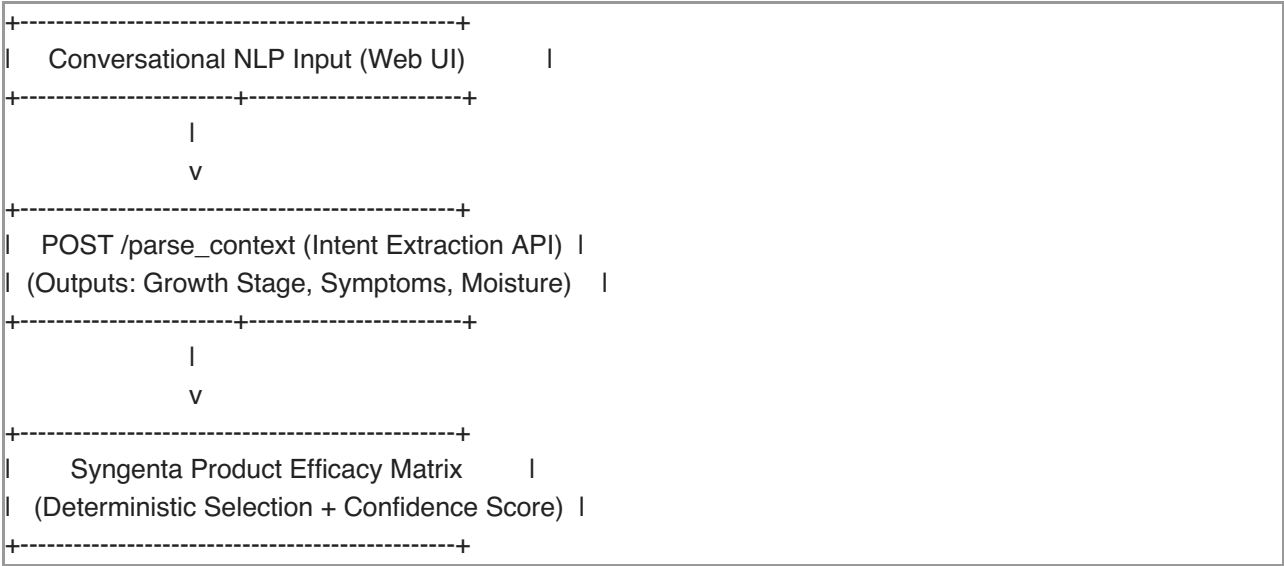
When Heat and Drought stress co-occur, they destroy yield exponentially faster than in isolation. The engine applies a synergistic coefficient ($k=1.2$):

$$\text{Compound Score} = \min(1.0, (\text{Heat} + \text{Drought}) \times 1.2)$$

Part 2: Problem Statement 03 (PS-03) — CropFit Advisor

2.1 Architectural Overview & Logic Flow

PS-03 resolves the specific challenge of matching biological products to localized, in-field conditions. It replaces guesswork with deterministic mapping driven by a natural language interface.



2.2 Conversational AI Intent Parsing

The frontend provides an "Ask AI Advisor" text area where farmers describe their field conditions in plain English (e.g., "My plants are wilting and the soil is bone dry.").

The backend /parse_context API operates as a Multi-Modal Intent Extractor. It scans the unstructured payload to isolate discrete variables:

- **Symptoms Extraction:** Triggers on ["wilt", "dry", "droop"] $\rightarrow$ Wilting
- **Moisture Extraction:** Triggers on ["waterlog", "mud", "flood"] $\rightarrow$ Waterlogged
- **Phenology Extraction:** Triggers on ["flower", "bloom"] $\rightarrow$ Flowering

2.3 The Syngenta Decision Matrix (product_matrix.py)

Once the context is structured, it passes into the CropFit recommendation matrix. This engine evaluates exact permutations to output the correct Syngenta Biological alongside a dynamic Confidence Score and plain-language Rationale.

Condition A: Acute Drought Rescue

- **Logic:** IF Symptoms == Wilting AND Soil == Dry
- **Action:** Deploy **Quantis®**
- **Confidence Base:** \$92%\$
- **Rationale:** Recommends an osmo-protectant to preserve cell turgor pressure and mitigate heat-shock protein degradation.

Condition B: Fungal Pathogen Prevention

- **Logic:** IF Soil == Waterlogged (Overrides all other symptoms due to severe risk)
- **Action:** Deploy **Amistar Top®**
- **Confidence Base:** \$88%\$
- **Rationale:** Recommends immediate broad-spectrum fungicide application to preempt root rot and foliar blight in anaerobic soil conditions.

Condition C: Nutrient Lockout Correction

- **Logic:** IF (Symptoms == Stunting OR Yellowing) AND Soil == Optimal
- **Action:** Deploy **Coucal®**
- **Confidence Base:** \$80%\$
- **Rationale:** Diagnoses that the plant is failing to uptake available nutrients. Recommends biological stimulation of root efficacy.

2.4 Outcome Feedback Loop

To fulfill the PS-03 mandate for continuous learning, the UI injects a binary feedback mechanism ([Yes] [No]) below every CropFit recommendation. This telemetry closes the loop, allowing future model iterations to calibrate their confidence scores based on empirical yield outcomes.

Part 3: System API & Execution Standards

3.1 Backend Endpoints

- POST /run_pipeline: Accepts {region, crop_type, growth_stage, symptoms, soil_moisture}. Executes the full 14-day weather ingestion, hybrid mechanistic scoring, and CropFit matrix evaluation in a single non-blocking thread.
- POST /parse_context: Natural language parser.
- GET /get_regions: Returns pre-cached geo-spatial and soil metadata.

3.2 Resilience & State Management

- **Zero-Dependency Core:** The matrix logic and alert engines are built on standard libraries to guarantee sub-100ms execution times without reliance on heavy machine-learning frameworks in production.
- **Stateless Architecture:** The Flask application maintains no session state, ensuring infinite horizontal scalability for tens of thousands of concurrent farmer requests during critical weather events.